CE 502 · ADVANCED TOPICS IN DATABASE SYSTEMS

Week 01

Why Databases Matter & Transaction Logging

Why organisations cannot afford to lose a database, the careers built around protecting data, and the log that lets a DBMS survive a crash.

Dr. Afan HASAN · [Semester]

THIS WEEK

1-1	**Course Introduction and Careers** 40 minutes
1-2	**Database Management Challenges** 40 minutes
1-3	**Transaction Logging** 45 minutes
1-4	**Transaction Control** 45 minutes
P1	**Practice: watching the WAL** 90 minutes · PostgreSQL lab

CE 502 · WEEK 1 · LESSON

1-1

Course Introduction and Careers

Lesson 1 of 4

In this lesson

01 What this course is about and how it is organised

02 How the course works and is assessed

03 Where data professionals work

04 Six careers in database technologies

40 minutes

Builds on: SQL and relational database design

Objectives

By the end of this lesson, you will be able to:

01 **Describe** the purpose and four-block structure of CE 502

Checked in: Quiz 1 · question 1

02 **Explain** how the course is delivered and assessed

Checked in: Quiz 1 · question 2

03 **Identify** where data professionals work inside an organisation

Checked in: Quiz 1 · question 3

04 **Compare** six database careers and the course topics each one needs

Checked in: Careers assignment and Check Your Understanding

The chip in the top-right corner of each slide shows which objective that slide serves.

Objective 01

Why This Course?

- Your first database course taught you to design tables and write SQL. This course explains what happens **inside** and **around** the database in real use.
- Real systems must survive crashes, serve many users at once, run on many machines, support analytics and now power AI applications.
- By the end of the course you will be able to choose, design and operate the right kind of database for each situation.

Five questions this course answers

- How does a database keep data safe after a crash?
- How do many users change the same data at once?
- How is one database spread over many sites?
- How do we analyse very large amounts of data?
- How do databases power AI search?

Key point: using a database and running one are different skills. This course is about running one.

Objective 01

The 12-Week Course Map

The course has four blocks. Each block builds on the one before it.

Weeks 1–3

One database

Logging, backup and recovery, replication, concurrency control

Keep one database correct and safe

Weeks 4–6

Distributed

Architectures, horizontal and vertical fragmentation

Spread a database over many sites

Weeks 7–9

Analytics & scale

Data warehouses and cubes, big data, NoSQL

Analyse and scale very large data

Weeks 10–12

AI-era databases

Embeddings, vector search, RAG and recommenders

Build databases that power AI

Key point: you cannot distribute a database you do not understand, and you cannot scale what you cannot distribute.

Objective 02

How the Course Works

Each week has three or four short lessons, a hands-on practice, a short quiz and two or three research papers to read.

Component	What it is	Weight
Practices	Weekly labs in Docker: PostgreSQL, MongoDB, Cassandra, vector databases	[__%]
Quizzes	15–20 questions after each week’s lessons	[__%]
Assignments	Careers, data warehouse, big data and NoSQL briefs	[__%]
Midterm exam	Weeks 1–6, after Week 6	[__%]
Final project	Team RAG application or recommender, Week 12	[__%]

← **Every week has a lab.** Install Docker and PostgreSQL 16 before Practice 1.

← **Weights** are confirmed in the syllabus at the start of term.

Weekly rhythm: lessons → practice lab → quiz → papers to read before the next week.

Objective 04

Know why you are here before you learn how.

Motivation decides how much effort you are willing to put into hard topics such as recovery algorithms or fragmentation design. People who know their purpose choose roles that suit them and keep learning after their degree.

- Activity: finish the sentence “In five years I want to be a professional who …”. Keep it for Week 12.

Objective 03

Where Data Professionals Work

CIO
Runs IT as a business service

CTO
Technology strategy and architecture

CISO
Security, risk and compliance

CDO
Data strategy and governance

← **The CDO** shows that organisations now treat data as a strategic asset.

Data management	Operations	Application software
Networks	Security	Projects, training, support

← **Most database jobs** are in the three highlighted departments.

• Organisations group their IT staff under a small team of executives. Knowing this structure tells you who you would report to in each role.

Objective 04

Six Careers in Database Technologies: The Classic Roles

Role	Main responsibilities	Typical skills	Key weeks
Database administrator (DBA)	Installs and upgrades the DBMS, runs backup and recovery, tunes performance, controls access	SQL, scripting, monitoring, recovery tools	1–3, 9
SQL / database developer	Designs schemas and writes queries, views and stored procedures in an application team	SQL, PL/pgSQL or T-SQL, query tuning	1–3, 7
Data architect	Designs data models, sets standards and governance, chooses data platforms	Data modelling, cloud platforms, governance	4–7, 9

Read the last column. Every role needs Weeks 1–3: all of them must keep data correct.

Key point: every classic role starts from SQL and a database that stays correct, fast and recoverable.

Objective 04

Six Careers in Database Technologies: The Newer Roles

Role	Main responsibilities	Typical skills	Key weeks
BI analyst / data scientist	Builds dashboards and reports, mines data, analyses trends and forecasts	SQL, Python or R, BI tools, statistics	7–8
Data engineer	Builds pipelines that collect, clean, move and store data at scale	Python, SQL, Spark, Kafka, cloud services	4–9
AI / vector database engineer	Creates embeddings, runs vector search, builds RAG and recommenders	Python, embeddings, pgvector or vector DBs	10–12

← **All three still use SQL every day**, even though their tools are newer.

Key point: the newer roles add Python, pipelines and embeddings on top of the same database foundations.

Objective 04

Case Study: Choosing a Role

THE SITUATION

Elena has worked for two years as a junior developer in a retail company, writing SQL for the online shop and helping the DBA with backups. She reads this job advert:

> "We are hiring a **Data Engineer** to move data from our PostgreSQL order database into a cloud warehouse, keep the pipelines reliable, and support our new AI product-search team."

YOUR TASK · PAIRS · 6 MIN

1. Which skills in the advert does Elena already have?
2. Which skills must she still learn?
3. Which weeks of CE 502 would help her most?
4. What could her next role be after this one?

After discussing in pairs, compare your answers with the model answer in the reading notes.

Objective 04

Careers Assignment: Map One Career Path

- Choose one of the six roles and find **three real job adverts** for it, published in the last six months.
- Write 600–800 words: daily responsibilities, required skills and certifications, where this course's topics appear, and a 12-month learning plan.
- Review three classmates' reports with the rubric. Report due in **Week 3**: [date].

Rubric · 20 points

- Evidence: three cited adverts · 4
- Role description · 4
- Skills and certifications · 4
- Link to CE 502 topics · 4
- Learning plan and writing · 4

⚠ **Common mistake:** describing a role from memory. Use real adverts, cite each one, and quote the requirements they actually list.

Objective 04

Check Your Understanding

A job advert asks for someone to “design the data model for a new customer platform, set naming and data-quality standards, and choose between PostgreSQL and a cloud warehouse”. Which role is it?

A Database administrator

B Data architect

C Data engineer

D BI analyst

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

DBMS

Database management system: software that stores, retrieves and protects data, such as PostgreSQL.

Database developer

Designs schemas and writes the SQL code of an application.

Data engineer

Builds pipelines that collect, clean and move data at scale.

Chief Data Officer

Executive responsible for data strategy and governance.

Database administrator

Keeps databases available, fast, secure and recoverable.

Data architect

Designs data models and standards and chooses data platforms.

AI / vector DB engineer

Builds embeddings, vector search and RAG or recommender systems.

Data governance

Rules deciding who owns data, what it means and how it may be used.

Summary

In this lesson, you learned how to:

- **01** Describe the purpose and four-block structure of CE 502
- **02** Explain how the course is delivered and assessed
- **03** Identify where data professionals work inside an organisation
- **04** Compare six database careers and the course topics each one needs

Key takeaway: every database career depends on the same foundation: keeping data correct, available and secure. That is where this course starts.

Next lesson

1-2 Database Management Challenges

CE 502 · WEEK 1 · LESSON

1-2

Database Management Challenges

Lesson 2 of 4

In this lesson

01 Databases that hold data about you

02 The cost of downtime and three failure modes

03 Why the database tier is the hardest to scale

04 Size, concurrent users and security

40 minutes

Builds on: basic web application concepts

Objectives

By the end of this lesson, you will be able to:

01 **Identify** databases that hold data about you and what stops when they are down

Checked in: Quiz 1 · question 4

02 **Estimate** the cost of downtime and distinguish three failure modes

Checked in: Quiz 1 · questions 5–6

03 **Describe** a three-tier architecture and why the database tier is hardest to scale

Checked in: Quiz 1 · question 7

04 **Describe** the challenges of data size, concurrent users and security

Checked in: Quiz 1 · question 8

The chip in the top-right corner of each slide shows which objective that slide serves.

Objective 01

Databases That Hold Data About You

Most of the systems you use every day are applications built on top of a database.

- Each organisation you deal with keeps records about you in one or more databases.
- The application you see (a website, an app, a card terminal) is only the front end. The records live in the database behind it.
- If the database is unavailable, the application cannot do its job, however well the rest of it works.

Who keeps data about you?

- **University:** enrolment, grades, fees
- **Employer:** contract, salary, leave
- **Government:** tax, identity, vehicles
- **Mobile operator:** calls, data use, location
- **Bank:** accounts, cards, every payment
- **Online services:** orders, messages, history

Key point: databases are not a background technology. They run everyday life, so their failures are felt by real people.

When the Database Is Down

What stops in each organisation when its main database is unavailable?

System	What stops when the database is down
University	Registration, timetables, publishing grades
Employer	Payroll run, leave requests, access badges
Government	Tax returns, passport and licence checks
Mobile operator	Billing, top-ups, new SIM activation
Bank & card network	Card payments, transfers, cash withdrawals
Hospital	Patient records, prescriptions, test results

← Not just “the website is down”: the **business process itself** stops.

← Staff often fall back on paper, which is slow and must be re-entered later.

Key point: a database outage is a business outage. The process stops, not just the screen.

Objective 02

Worked Example: The Cost of One Hour of Downtime

An online retailer earns about €1,000,000 a day from sales that all pass through its order database.

Step	Calculation	Result
Daily revenue	given	€1,000,000
Hours per day	sales arrive around the clock	24
Revenue per hour	€1,000,000 ÷ 24	≈ €41,700
Direct loss, 1 hour down	no orders can be taken	≈ €41,700

← This is a **lower bound**. Indirect losses are often larger.

← Peak hours (evenings, sales days) cost far more than the average.

⚠ **Indirect losses:** customers who never return, staff overtime, contract penalties, fines and damage to reputation.

Objective 02

Three Failure Modes: Down, Slow, Crashed

Down

Users see: no response at all; every request fails

Typical causes: hardware failure, network loss, power cut, maintenance

Protection: replication and failover · Weeks 2, 4

Slow

Users see: responses that take seconds instead of milliseconds

Typical causes: heavy queries, too many users, locking, missing indexes

Protection: tuning, scaling, concurrency control · Weeks 3, 5

Crashed

Users see: the server stops in the middle of work

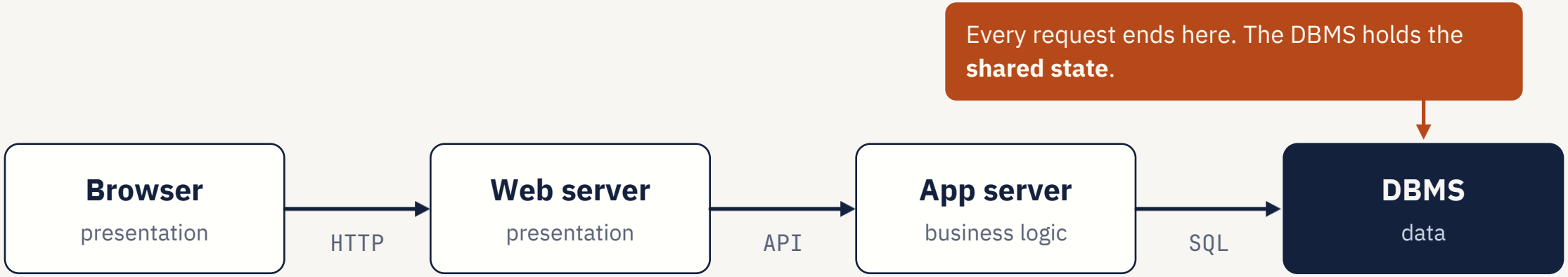
Typical causes: software fault, power cut, out of memory or disk

Protection: transaction log, backup and recovery · Weeks 1, 2

⚠ **Common mistake:** treating a slow database as “still working”. If a payment takes 30 seconds, customers give up and the business loses the sale just as if the database were down.

Objective 03

Where the Database Sits: Three-Tier Architecture



Copies can be added freely: 1 → 10 → 100 servers behind a load balancer.

- The first tiers only pass requests along and can be copied onto many servers.
- All copies must read and write the **same** data, so they all depend on the one database tier.

Objective 03

Why the Database Tier Is the Hardest to Scale

</> Web and app tiers: stateless

- Keep nothing between requests
- Any copy can serve any request
- Scale out: add identical servers
- A failed copy is simply replaced

Database tier: stateful

- Keeps every record permanently
- All copies must agree on the same data
- Each change must be safe on disk
- A failed server may hold the only copy

Key point: adding servers is easy when they hold no data. Once they share data, they must coordinate, and coordination is what makes databases hard.

Objective 04

Challenge 1: The Size of the Data

- Operational databases often hold several terabytes; analytical ones reach petabytes.
- Every routine task (copying, indexing, checking, restoring) takes longer as the data grows.
- Tasks that take minutes on a test database can take hours in production.

COPYING A 10 TB DATABASE

10 TB = 10,000,000 MB

÷ 500 MB/s = 20,000 s

≈ 5.6 hours

A restore takes **at least** as long as a copy, so a lost 10 TB database means hours of downtime. →

Objective 04

Challenges 2 and 3: Concurrent Users and Security

Concurrent users

Thousands of transactions run at the same moment, often on the **same rows**: two people booking the last seat, two cashiers selling the last item.

The DBMS must keep the data correct without making everyone wait.

Security and privacy

Databases hold personal data, so a single leak can expose millions of people.

Access must be limited, data encrypted, and every use traceable.

In practice

In the EU, the General Data Protection Regulation (GDPR) requires organisations to protect personal data and report serious breaches to the regulator, normally within 72 hours. Fines can reach 4% of worldwide annual turnover.

Objective 04

When a server crashes in the middle of thousands of changes, the database must remember every confirmed one and forget every unfinished one.

Every challenge in this lesson meets in this moment. The tool that makes it possible is the **transaction log**, the subject of Lesson 1-3.

Committed work survives (durability) · Unfinished work disappears (atomicity)

Objective 02

Case Study: Friday Evening at the Bank

THE SITUATION

A regional bank's card-payment database stops responding at **18:00 on a Friday** and is back after **40 minutes**. At that time it normally handles about **2,000 card payments a minute**.

> "The board wants to know what happened, what it cost us, and how we stop it happening again." (Head of Operations)

YOUR TASK · PAIRS · 6 MIN

1. Which failure mode is this: down, slow or crashed?
2. How many payments were affected?
3. List three indirect costs for the bank.
4. Which weeks of the course would help prevent it?

After discussing in pairs, compare your answers with the model answer in the reading notes.

Check Your Understanding

A large reporting query runs during business hours. For two hours, the customer website takes about **25 seconds** to respond instead of 1 second. No data is lost. Which failure mode is this, and what is the right kind of protection?

A Down: replication and failover

B Slow: tuning, scaling and concurrency control

C Crashed: backup and recovery

D No failure: the database still answered

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Availability

The proportion of time a system answers correctly and quickly enough to be used.

Downtime

A period when the system cannot be used, whether planned or unplanned.

Failure mode

The way a system fails: down, slow or crashed.

Three-tier architecture

Presentation, application logic and data tiers, each on its own servers.

Stateless

Keeps nothing between requests, so any copy can serve any request.

Stateful

Keeps data that must survive and stay consistent, like a database.

Concurrency

Many transactions running at the same time, often on the same data.

Data protection

Legal and technical duty to keep personal data secure and private.

Summary

In this lesson, you learned how to:

- **Identify** databases that hold data about you and what stops when they are down
- **Estimate** the cost of downtime and distinguish down, slow and crashed
- **Describe** a three-tier architecture and why the stateful database tier is hardest to scale
- **Describe** the challenges of data size, concurrent users and security

Key takeaway: after a crash, a database must keep all committed work and remove all unfinished work. The transaction log makes this possible.

Next lesson

1-3 Transaction Logging

CE 502 · WEEK 1 · LESSON

1-3

Transaction Logging

Lesson 3 of 4

In this lesson

01 Why every DBMS keeps a transaction log

02 The write-ahead rule and log flushing

03 Reading log records and a sample log

04 Checkpoints and crash recovery

45 minutes

Builds on Lesson 1-2: database challenges

Objectives

By the end of this lesson, you will be able to:

01 **Explain** why every DBMS keeps a transaction log and what the log makes possible

Checked in: Quiz 1 · question 9

02 **Describe** the write-ahead rule and when the log buffer is flushed to disk

Checked in: Quiz 1 · question 10

03 **Identify** the fields of a log record and read a sample log

Checked in: Quiz 1 · question 11 and Practice 1

04 **Explain** how checkpoints and the log are used in crash recovery

Checked in: Quiz 1 · question 12

The chip in the top-right corner of each slide shows which objective it serves.

Objective 01

Why Every DBMS Keeps a Transaction Log

- Every change made to a database is first recorded in a **transaction log**: a history of what changed, by which transaction, and when.
- Data in memory is lost in a crash, and data files on disk may be only partly updated. The log is the record the DBMS trusts.
- One log serves many purposes, from undoing a single failed statement to rebuilding an entire database.

What the log makes possible

- Rollback of failed work (1-4)
- Crash recovery (this lesson)
- Point-in-time recovery (Week 2)
- Replication to a standby (Week 2)

Key point: the log is not a copy of the database. It is the history of changes, and with it the DBMS can move the data forward or backward in time.

Objective 01

The Log in Different Database Systems

DBMS	Name of the log	Tool to read it
PostgreSQL	Write-ahead log (WAL)	pg_waldump
SQLite	Write-ahead log (WAL mode)	Built-in WAL file
MySQL	Binary log + InnoDB redo log	mysqlbinlog
Oracle Database	Online and archived redo logs	LogMiner
SQL Server	Transaction log (.ldf)	fn_dblog()

Note MySQL: two logs with two jobs. The InnoDB redo log handles crash recovery; the binary log feeds replication and point-in-time recovery.

The names differ, but the idea is the same everywhere: record each change before it reaches the data files.

Objective 01

What the Log Looks Like

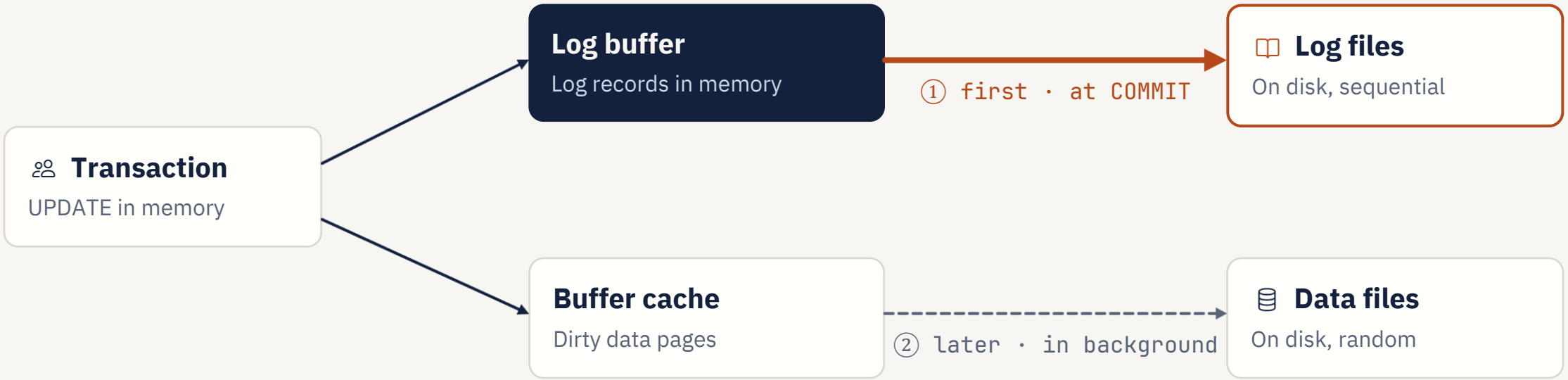
- The log is a **sequential**, append-only file: new records are always added at the end, never in the middle.
- The log is **binary**, not human-readable. Each DBMS provides a utility that prints it in readable form.
- Every record has a **log sequence number** (LSN), its position in the log. LSNs only increase, so they show the order of changes.

```
$ pg_waldump (shortened output)
1 rmgr: Heap tx: 742 lsn: 0/01A3F2C8
2 desc: UPDATE off 7 xmax 742
3 rmgr: Transaction tx: 742
4 lsn: 0/01A3F310 desc: COMMIT
```

Read this line: the COMMIT record of transaction 742, at its own LSN.

Objective 02

Two Paths to Disk: The Log and the Data Files



Critical point: the two paths are written at different times. COMMIT waits only for path ①; path ② can happen seconds or minutes later.

Objective 02

Write the log first. Write the data later.

The write-ahead rule: a changed data page may never reach the data files before the log records that describe its changes are safely on disk.

This one rule is why a crash can never leave changes that the log does not know about.

Objective 02

When the Log Buffer Is Flushed

Writing the log buffer from memory to the log file on disk is called a flush.

Trigger	Why it happens
COMMIT	Required: a commit is durable only when its records are on disk
Timer	Every few hundred milliseconds, to limit what a crash can lose
Buffer full	The log buffer reached its size limit
Manual command	A DBA forces it, e.g. FLUSH LOGS in MySQL

← **The trigger that matters:** COMMIT. Without it, a confirmed transaction could disappear in a crash.

Rotation

Full log files are closed and the next number is opened: binlog.000041 → binlog.000042.

Objective 03

Anatomy of a Log Record

Field	Purpose
LSN	Unique, increasing position of the record in the log
Transaction ID	Which transaction made the change
Previous LSN	This transaction’s previous record (a backward chain)
Object	The table, row or page that was changed
Operation	BEGIN, INSERT, UPDATE, DELETE, COMMIT, ABORT
Before image	The old value: used to undo
After image	The new value: used to redo

Why both images? With the old value the DBMS can undo; with the new value it can redo. Every recovery decision uses one of the two.

Objective 03

Reading a Sample Log

An invented log of three concurrent transactions. The server crashes right after record 109.

LSN	Tx	Prev	Object	Operation	Before	After
101	T1	–	–	BEGIN	–	–
102	T1	101	ACCOUNT 7	UPDATE	bal=500	bal=380
103	T2	–	–	BEGIN	–	–
104	T2	103	ITEM 500	UPDATE	qty=120	qty=132
105	T1	102	ACCOUNT 9	UPDATE	bal=200	bal=320
106	T1	105	–	COMMIT	–	–
107	T3	–	–	BEGIN	–	–
108	T3	107	EMP 54321	UPDATE	comm=0	comm=120
109	T2	104	–	COMMIT	–	–

Interleaved: T1, T2 and T3 run at the same time. Prev LSN links each one’s records.

T3 has no COMMIT. Its change at 108 must not survive the crash.

Objective 03

Scenario: Recover the Sample Log

THE SITUATION

The server lost power right after LSN 109 of the sample log. It has just restarted, and the DBMS is reading the log on disk.

> “Customers saw ‘transfer complete’ for T1. Is their money safe?” — the bank’s operations manager

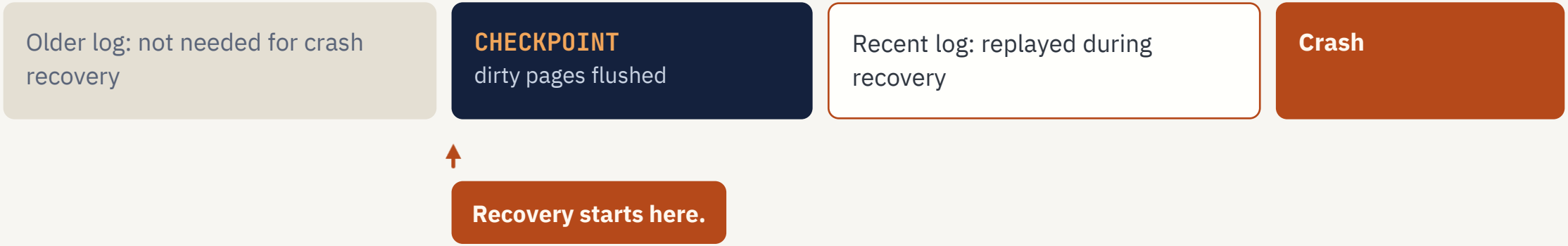
You may assume any of the changed pages did or did not reach the data files.

YOUR TASK · PAIRS · 6 MIN

1. Which transactions had committed?
2. Which changes must be redone, and which undone?
3. What are the final values of account 7, account 9, item 500 and employee 54321?
4. What do you answer the manager?

Checkpoints Keep Recovery Short

- A **checkpoint** writes all dirty pages from memory to the data files, then records a CHECKPOINT entry in the log.
- After a crash, recovery starts reading the log at the **last checkpoint**, not at the beginning of time.



Key point: frequent checkpoints mean shorter recovery but more disk writes; rare checkpoints mean the opposite. It is a tuning trade-off.

Objective 04

Going Deeper: Crash Recovery in Three Phases

Most modern DBMSs recover with a method based on ARIES (Mohan et al., 1992).

1 · Analysis

Read forward from the last checkpoint. Find which transactions were active and which pages may be out of date.

2 · Redo

Replay every logged change, for all transactions, until the data matches the log exactly: “repeat history”.

3 · Undo

Roll back the transactions that never committed, following their previous-LSN chains backwards.

Why redo T3 first, then undo it? Repeating history makes the state exactly what existed at the crash, so undo always starts from a known state.

Objective 02

Check Your Understanding

A transaction updates three rows and its COMMIT is confirmed. One second later the server loses power. What must be on disk at the moment the COMMIT was confirmed?

A The three changed data pages

B The transaction’s log records, including COMMIT

C Both the data pages and the log records

D Nothing yet; everything is still in memory

Think for 30 seconds, then vote: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Transaction log

A sequential record of every change to the database, used for undo, redo and recovery.

Log sequence number (LSN)

The unique, increasing position of a record in the log.

Dirty page

A data page changed in memory but not yet written to the data files.

Checkpoint

A point where dirty pages are written to disk; recovery starts here.

Write-ahead log (WAL)

A log that must reach disk before the data pages it describes.

Log buffer / flush

The in-memory area for log records / writing it to the log file on disk.

Before / after image

The value before a change (for undo) / after it (for redo).

Redo / undo

Reapplying committed changes / removing uncommitted ones.

Summary

In this lesson, you learned how to:

- **01** Explain why every DBMS keeps a transaction log and what it makes possible
- **02** Describe the write-ahead rule and when the log buffer is flushed
- **03** Identify the fields of a log record and read a sample log
- **04** Explain how checkpoints and the log are used in crash recovery

Key takeaway: the log is written first and the data files later; after a crash, the log decides what is redone and what is undone.

Next

1-4 Transaction Control

CE 502 · WEEK 1 · LESSON

1-4

Transaction Control

Lesson 4 of 4

In this lesson

01 What a transaction (logical unit of work) is

02 Atomicity: all or nothing

03 COMMIT, ROLLBACK and SAVEPOINT in SQL

04 Autocommit and a first look at ACID

45 minutes

Builds on Lesson 1-3: transaction logging

Objectives

By the end of this lesson, you will be able to:

01 **Define** a transaction (logical unit of work) and give business examples

Checked in: Quiz 1 · question 13

02 **Explain** atomicity and the effect of COMMIT and ROLLBACK

Checked in: Quiz 1 · question 14

03 **Write** SQL that uses BEGIN, COMMIT, ROLLBACK and SAVEPOINT

Checked in: Practice 1 · Part C

04 **Describe** autocommit and link atomicity and durability to the log

Checked in: Quiz 1 · question 15

The chip in the top-right corner of each slide shows which objective it serves.

Objective 01

A Transaction Is a Logical Unit of Work

- A **transaction** is a sequence of reads and writes that together perform **one business action**.
- It is also called a **logical unit of work (LUW)**: the database is correct before it starts and after it ends, but not necessarily in between.
- The DBMS treats the whole sequence as one indivisible unit.

One business action, many changes

- Pay a salary: 3 ledger rows
- Place an order: 4 tables
- Transfer money: 2 accounts
- Book a seat: seat + ticket

Key point: the boundaries of a transaction are a business decision. The DBMS cannot guess which statements belong together; the application must tell it.

Objective 02

All or nothing.

Atomicity: either every change of a transaction becomes permanent, or none of them does. There is no third outcome, even if the server crashes halfway.

- The undo information in the log is what makes "nothing" possible.

Objective 02

COMMIT and ROLLBACK

COMMIT

Ends the transaction and makes all its changes **permanent**.

The COMMIT record is forced to the log on disk; other sessions can now see the changes.

ROLLBACK

Ends the transaction and **cancels** all its changes.

The DBMS uses the before images in the log to restore every changed row.

⚠ **Common mistake:** thinking that a confirmed COMMIT can be undone with ROLLBACK. Once committed, a change can only be reversed by a **new** transaction that changes the data back.

Example: Paying a Salary

One payment of 2,000 EUR touches three places in the accounting system.

Step	Change	Amount
1	Insert a row in the cheque register	2,000 EUR
2	Debit the cash account	− 2,000 EUR
3	Credit salaries payable	+ 2,000 EUR
✓	COMMIT: the books balance again	

Crash after step 2? Cash is lower but nothing is owed less. Recovery undoes steps 1–2, because there is no COMMIT.

Why not three transactions? Each one alone leaves the books wrong.

In practice

accounting systems check at the end of every transaction that total debits equal total credits. Atomicity guarantees that a half-posted payment can never be seen.

Objective 02

Example: An Order That Fails Halfway

Customer 1004 orders goods worth $1,200; the salesperson earns 10% commission.

#	Statement	Result
1	UPDATE customer 1004: balance + $1,200	done
2	UPDATE salesperson 54321: commission + $120	done
3	INSERT INTO "ORDER" ...	ERROR: table full
4	INSERT order lines; reduce stock	never run

Critical: without ROLLBACK, the customer is charged and the salesperson paid for an order that does not exist.

Key point: the application catches the error and issues ROLLBACK. Statements 1 and 2 are undone; the customer can try again later.

Objective 03

Controlling a Transaction in SQL

```
-- psql · PostgreSQL
1 BEGIN;
2 UPDATE account SET bal = bal - 120 WHERE id = 7;
3 UPDATE account SET bal = bal + 120 WHERE id = 9;
4 SAVEPOINT after_transfer;
5 UPDATE account SET bal = bal - 2.50 WHERE id = 7;
6 ROLLBACK TO SAVEPOINT after_transfer;
7 COMMIT;
```

1 starts the transaction.

2–3 transfer 120.

← **6 · Note:** cancels only the 2.50 fee; the transfer stays.

7 makes the transfer permanent.

⚠ **Common mistake:** ROLLBACK TO SAVEPOINT does **not** end the transaction. You still need COMMIT or ROLLBACK afterwards.

Objective 04

Autocommit: Every Statement Is a Transaction

Tool or driver	Autocommit by default	To group statements
psql, MySQL client	On	BEGIN … COMMIT
JDBC (Java)	On	setAutoCommit(false)
Python psycopg	Off	conn.commit()
Oracle SQL*Plus	Off	COMMIT

Danger: with autocommit on, a two-statement transfer is two transactions. A crash between them loses money.

- In **autocommit** mode, each statement is committed as soon as it succeeds. Know your tool’s default before you write a multi-statement change.

Objective 04

A First Look at ACID

Four properties every transaction should have. Two of them come directly from the log.

Property	Meaning	How it is achieved
Atomicity	All or nothing	Undo with before images
Consistency	Correct state to correct state	Constraints + correct transactions
Isolation	Concurrent transactions do not interfere	Locking, MVCC (Week 3)
Durability	Committed work survives failures	Redo with after images

← **Highlighted:** the two properties you can already explain with the log.

Key point: consistency and isolation are the topic of Week 3. Atomicity and durability you already understand from Lessons 1-3 and 1-4.

Objectives 02 · 04

Scenario: The Missing Money

THE SITUATION

A Java service transfers money with two UPDATE statements on a JDBC connection. The service was killed during a deployment.

> “Account 7 lost 120, but account 9 never received it. How is that possible?” — support team lead

YOUR TASK · PAIRS · 6 MIN

1. What is the most likely cause?
2. Why did the DBMS not undo the first UPDATE?
3. Fix the code: which three calls are needed?
4. How do you repair account 7 now?

Objective 03

Check Your Understanding

A session runs: BEGIN; INSERT row A; SAVEPOINT s1; INSERT row B; ROLLBACK TO SAVEPOINT s1; INSERT row C; COMMIT; — Which rows are in the table afterwards?

A A, B and C

B A and C

C Only A

D None: ROLLBACK cancelled everything

Think for 30 seconds, then vote: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Transaction / LUW

A sequence of operations performing one business action as a single unit.

COMMIT

Ends a transaction and makes its changes permanent and visible.

Savepoint

A named marker inside a transaction for a partial rollback.

Compensating transaction

A new transaction that reverses a committed change.

Atomicity

All changes of a transaction become permanent, or none do.

ROLLBACK

Ends a transaction and cancels all its changes.

Autocommit

A mode in which each statement is committed on its own.

ACID

Atomicity, consistency, isolation, durability.

Summary

In this lesson, you learned how to:

- **01** Define a transaction (logical unit of work) and give business examples
- **02** Explain atomicity and the effect of COMMIT and ROLLBACK
- **03** Write SQL with BEGIN, COMMIT, ROLLBACK and SAVEPOINT
- **04** Describe autocommit and link atomicity and durability to the log

Key takeaway: a transaction is defined by the application and protected by the DBMS. The log's before images give atomicity; its after images give durability.

Next

Practice 1: Inside the WAL

Objectives 1-3 · 1-4

Practice 1: Watch the Log at Work

Individual · about 90 minutes · PostgreSQL 16 in the course Docker container

Part A · steps 1–3

Measure the log

How much WAL does a bulk insert of 100,000 rows produce, compared with the table itself?

Part B · step 4

Read the log

Find your own transaction and its COMMIT record with pg_waldump; see full-page images after a CHECKPOINT.

Part C · steps 5–6

Break a transaction

Run a failing order inside BEGIN … ROLLBACK and prove that nothing remains.

Before you start

you need only the course repository and Docker. Every command is in **labs/week01/README.md**; copy them from there rather than retyping from the slides.

Measure How Much Log an Insert Produces

```
-- terminal and psql
1 docker compose up -d postgres
2 SELECT pg_current_wal_lsn(); -- e.g. 0/1A2B3C8
3 INSERT INTO lab1 SELECT g, md5(g::text) FROM generate_series(1,100000) g;
4 SELECT pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), '0/1A2B3C8'))
5 SELECT pg_size_pretty(pg_relation_size('lab1'));
```

1 Start the container and note the current WAL position (line 2).

2 Insert 100,000 rows (line 3).

← **3 · Critical:** compare WAL size (line 4) with table size (line 5).

⚠ **Common mistake:** pasting the example LSN on line 4. Use the value **your** line 2 printed, or the difference is meaningless.

Parts B and C · Steps 4–6

Read the Log, Then Break a Transaction

```
-- psql and container shell
1 BEGIN; SELECT txid_current(); UPDATE lab1 SET v='x' WHERE id=1; COMMIT;
2 pg_waldump -x 742 $PGDATA/pg_wal/000000010000000000000XX
3 CHECKPOINT; UPDATE lab1 SET v='y' WHERE id=2; -- then pg_waldump again
4 BEGIN; UPDATE stock ...; UPDATE customer ...;
5 INSERT INTO orders VALUES (1, NULL); -- ERROR: not-null violation
6 ROLLBACK; SELECT * FROM stock WHERE item=500;
```

4 Lines 1–3: find your transaction's records and its COMMIT; look for FPW after CHECKPOINT.

← **5 · Critical:** lines 4–6: the failed INSERT forces ROLLBACK.

6 Show that stock and customer are unchanged.

Key point: use **your** transaction ID from line 1 after -x on line 2, and your current WAL file name from pg_walfile_name(pg_current_wal_lsn()).

Assessment

What to Submit

One PDF report, maximum 4 pages, due before the Week 2 lesson. 10 points.

Deliverable	Points
WAL size vs table size, with your explanation (steps 1–3)	3
Screenshot of your transaction and its COMMIT record (step 4)	2
Screenshot of a full-page image after CHECKPOINT (step 4)	2
Evidence that ROLLBACK left no changes (steps 5–6)	1
Explanation: what recovery does after a crash before ROLLBACK	2
Total	10

Most points go to explanations, not screenshots. Link each result to a concept.

CE 502 · WEEK 1

Papers to Read

Three required papers behind this week’s lessons, three optional ones, and a method for reading them.

3 required · 3 optional

Before the Week 2 lesson

How to Read a Paper: Three Passes

Adapted from Keshav, “How to Read a Paper”, 2007.

Pass 1 · 10 min

Get the idea

Title, abstract, introduction, headings, conclusion. What problem? What contribution?

Pass 2 · 1 hour

Understand the content

Read the listed sections carefully. Study figures and examples. Note what you do not understand.

Pass 3 · optional

Recreate it

Re-derive the argument yourself. Only for papers central to your thesis or project.

▻ Your half-page note: the problem, the key idea, and the link to this week.

Required

Required Reading for Week 1

A Relational Model of Data for Large Shared Data Banks E. F. Codd · Communications of the ACM 13(6), 1970	The paper that founded relational databases. Read sections 1.1–1.4.
Principles of Transaction-Oriented Database Recovery T. Haerder, A. Reuter · ACM Computing Surveys 15(4), 1983	Coins “ACID”; classifies failures and logging. Read sections 1–3.
ARIES: A Transaction Recovery Method ... C. Mohan et al. · ACM TODS 17(1), 1992	Analysis, redo, undo: how real DBMSs recover. Read sections 1–2 and the overview.

Bring a half-page note per paper to the Week 2 lesson.

Optional

Optional Reading

The Transaction Concept: Virtues and Limitations
J. Gray · VLDB 1981

The classic, readable introduction to transactions. Short and clear.

What Goes Around Comes Around… And Around…
M. Stonebraker, A. Pavlo · SIGMOD Record 53(2), 2024

Fifty years of data models; why relational keeps winning.

How to Read a Paper
S. Keshav · ACM SIGCOMM CCR 37(3), 2007

Two pages. The three-pass method from the previous slide.

☆ Recommended if you are considering a thesis topic in databases.

Self-check

Reading Questions

Q1 **Codd:** which three kinds of data dependence did programs suffer from before the relational model?

Q2 **Haerder and Reuter:** name the three types of failure and the log information each one needs.

Q3 **ARIES:** why does redo “repeat history”, even for transactions that will be undone?

Q4 **Link:** which ARIES ideas can you see in the pg_waldump output of Practice 1?

Answer first, then compare with the reading text of this slide.

CE 502 · WEEK 1

This Week’s Work

Practice 1 report: Inside the WAL
One PDF, max 4 pages · 10 points — Before Week 2

Quiz 1: Lessons 1-1 to 1-4
Online, 15 questions, 25 minutes — Before Week 2

Careers assignment: database roles
Three job adverts, 600–800 words — Week 3

Required papers: Codd, Haerder and Reuter, ARIES
Half-page note per paper — Before Week 2

Next week: backup, recovery and replication: the log, archived and shipped.